

Лекция 6

Наследование

1 Базовый и производный классы

Наследование — процесс создания новых классов, называемых *наследниками* или *производными*, из уже существующих, базовых классов



Синтаксис:

```
class имя_производного_класса : режим_доступа  
    имя_базового_класса  
    { ... };
```

Режим доступа	Доступ в родительском классе	Доступ в дочернем классе
<i>public</i> (общий)	private	недоступен
	protected	protected
	public	public
<i>protected</i> (защищенный)	private	недоступен
	protected	protected
	public	protected
<i>private</i> (приватный)	private	недоступен
	protected	private
	public	private

```

1  #include <iostream>
2  using namespace std;
3
4  class Person
5  {
6      protected:
7          string name;    //  ИМЯ
8          int age;        //  ВОЗРАСТ
9      public:
10         void Set_Person(string name1, int age1)
11         { name = name1; age =age1; }
12         void Show()
13         { cout << "Name: " << name << "\nAge: " << age << endl; }
14     };
15
16     class Employee : public Person
17     {
18         string company; //  КОМПАНИЯ
19     public:
20         void Set_company(string company1)
21         { company = company1; }
22         void Show_company()
23         { cout << "Company: " << company << endl; }
24     };
25
26     int main()
27     {
28         Employee Alex;
29         Alex.Set_Person("Alex", 25);
30         Alex.Set_company("IBM");
31
32         Alex.Show();
33         Alex.Show_company();
34     }

```

Name: Alex
Age: 25
Company: IBM

2 Конструкторы и наследование

```
1  #include <iostream>
2  using namespace std;
3
4  class Person
5  {
6  protected:
7      string name;    // имя
8      int age;        // возраст
9  public:
10     Person(): name("Bob"), age(20) {}
11     Person(string name1, int age1): name(name1), age(age1) {}
12 };
13
14 class Employee : public Person
15 {
16     string company; // компания
17 public:
18     Employee(): Person(), company("Google") {}
19     Employee(string name1, int age1, string company1):
20         Person(name1, age1), company(company1) {}
21     void Show()
22     { cout << "Name: " << name << "\nAge: " << age <<
23         "\nCompany: " << company << endl; }
24 };
25
26 int main()
27 {
28     Employee Bob;
29     Employee Alex("Alex", 25, "IBM");
30
31     Bob.Show();
32     Alex.Show();
33 }
```

Для производного класса надо создавать новые конструкторы, в которых можно использовать конструкторы родительского класса

Использование конструктора базового класса

```
Name: Bob
Age: 20
Company: Google
Name: Alex
Age: 25
Company: IBM
```

3 Перегрузка методов базового класса

```
1 #include <iostream>
2 using namespace std;
3
4 class Person
5 {
6     protected:
7         string name;    // имя
8         int age;        // возраст
9     public:
10         Person(): name("Bob"), age(20) {}
11         Person(string name1, int age1): name(name1), age(age1) {}
12         void Show()
13         { cout << "Name: " << name << "\nAge: " << age << endl; }
14 };
15
16 class Employee : public Person
17 {
18     string company;    // компания
19     public:
20         Employee():Person(), company("Google"){ }
21         Employee(string name1, int age1, string company1):
22             Person(name1, age1), company(company1){ }
23         // перегрузка метода Show() базового класса
24         void Show()
25         { // вызов метода Show() базового класса
26             Person::Show();
27             cout << "Company: " << company << endl; }
28 };
```

```
29
30 int main()
31 {
32     Person Ann;
33     Employee Bob;
34     Employee Alex("Alex", 25, "IBM");
35
36     Ann.Show();
37     Bob.Show();
38     Alex.Show();
39 }
```

Выполняется
метод Show
базового класса

Выполняется
метод Show
производного
класса
(перегруженный
метод базового
класса)

```
Name: Bob
Age: 20
Company: Google
Name: Alex
Age: 25
Company: IBM
```

4 Общее и частное наследование

Пример public наследования:

```
1  #include <iostream>
2  using namespace std;
3
4  class A // базовый класс
5  {
6  private:  int PrivDataA;
7  protected: int ProtDataA;
8  public:   int PubDataA;
9  };
10
11 class B : public A // public наследование
12 {
13 public:
14     void SetData()
15     {
16         // PrivDataA = 1; // ошибка, нет доступа к полю PrivDataA
17         ProtDataA = 2; // так можно (ProtDataA с доступом protected)
18         PubDataA = 3;  // так можно (PubDataA с доступом public)
19     }
20
21     void ShowData()
22     {
23         // cout << PrivDataA << endl; // ошибка, нет доступа к PrivDataA
24         cout << ProtDataA << endl << PubDataA << endl;
25     }
26 };
27
28 int main()
29 {
30     B b;
31     b.SetData();
32     b.ShowData();
33 }
```

2

3

Пример private наследования:

```
1  #include <iostream>
2  using namespace std;
3
4  class A // базовый класс
5  {
6  private:  int PrivDataA;
7  protected: int ProtDataA;
8  public:   int PubDataA;
9  };
10
11 class B : private A // private наследование
12 {
13 public:
14     void SetData()
15     {
16         // PrivDataA = 1; // ошибка, нет доступа к полю PrivDataA
17         ProtDataA = 2; // так можно (ProtDataA с доступом private,
18                        // для производных классов доступа к полю не будет)
19         PubDataA = 3; // так можно (PubDataA с доступом private,
20                      // для производных классов доступа к полю не будет)
21     }
22
23     void ShowData()
24     {
25         // cout << PrivDataA << endl; // ошибка, нет доступа к PrivDataA
26         cout << ProtDataA << endl << PubDataA << endl;
27     }
28 };
29
30 int main()
31 {
32     B b;
33     b.SetData();
34     b.ShowData();
35 }
```

2

3

Пример protected наследования:

```
1  #include <iostream>
2  using namespace std;
3
4  class A // базовый класс
5  {
6  private:  int PrivDataA;
7  protected: int ProtDataA;
8  public:   int PubDataA;
9  };
10
11 class B : protected A // protected наследование
12 {
13 public:
14     void SetData()
15     {
16         // PrivDataA = 1; // ошибка, нет доступа к полю PrivDataA
17         ProtDataA = 2; // так можно (ProtDataA с доступом protected,
18                        // для производных классов поле будет доступно)
19         PubDataA = 3; // так можно (PubDataA с доступом protected,
20                      // для производных классов поле будет доступно)
21     }
22
23     void ShowData()
24     {
25         // cout << PrivDataA << endl; // ошибка, нет доступа к PrivDataA
26         cout << ProtDataA << endl << PubDataA << endl;
27     }
28 };
29
30 int main()
31 {
32     B b;
33     b.SetData();
34     b.ShowData();
35 }
```

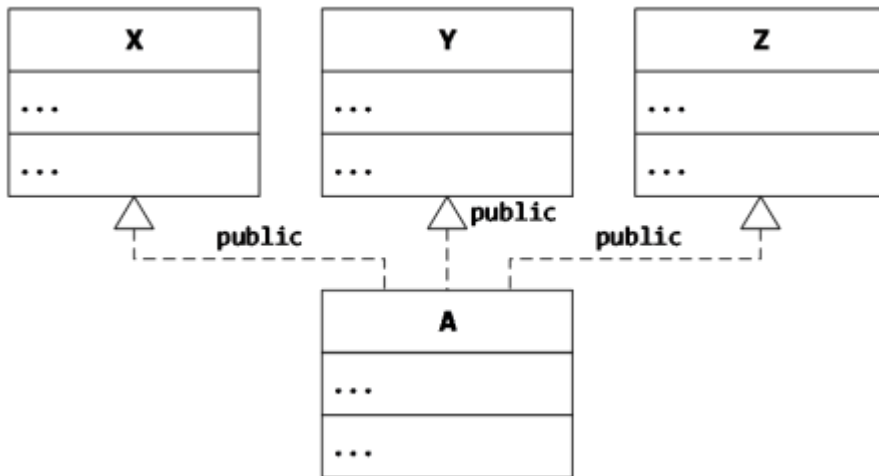
2

3

5 Множественное наследование

При множественном наследовании класс является производным от нескольких базовых классов

```
class имя_производного_класса: список_базовых_классов  
    { ... };
```



```
class X
{...};
class Y
{...};
class Z
{...};
class A : public X, public Y, public Z
{...};
```

Неопределенности при множественном наследовании?

6 Включение: классы в классах

В ООП включение появляется, когда один объект является атрибутом другого объекта. Например:

```
class A { ... };  
class B { ...  
    A objA; // полем класса является объект  
    ... };  
  
...  
B objB;  
...
```

```

1  #include <iostream>
2  using namespace std;
3
4  class Person
5  {
6      string name;    // ИМЯ
7      int age;        // ВОЗРАСТ
8  public:
9      void Set_Person(string name1, int age1)
10     {
11         name = name1;
12         age = age1;
13     }
14
15     void Show()
16     {
17         cout << "Name: " << name <<
18             "\nAge: " << age << endl;
19     }
20 };

```

```

Name: Alex
Age: 25
Company: IBM

```

```

21
22 class Employee : public Person
23 {
24     Person P;
25     string company; // КОМПАНИЯ
26 public:
27     Employee(string name1, int age1, string company1)
28     {
29         P.Set_Person(name1, age1);
30         company = company1;
31     }
32
33     void Set_Employee(string name1, int age1, string company1)
34     {
35         P.Set_Person(name1, age1);
36         company = company1;
37     }
38
39     void Show()
40     {
41         P.Show();
42         cout << "Company: " << company << endl;
43     }
44 };
45
46 int main()
47 {
48     Employee Alex("Alex", 25, "IBM");
49     Alex.Show();
50 }

```

7 Запрет наследования

Модификатор **final** используется для того, чтобы запретить наследование.

```
class A final
{
    ...
};

class B : public A // ОШИБКА!!!
{
    ...
};
```